



Título del proyecto: UrbanFlow – Sistema de Gestión de Pedidos e Inventario de Ropa

Asignatura:

Herramientas de Desarrollo

Integrantes:

Uriel Vasquez Quispe (U23227010)

Christian Rivas Aquino (U23226966)

Renzo Estefano Yupanqui De la Cruz (U22229407)

David Jair Mendoza Figueroa (U22226721)

Aldair Ronaldo Casimiro Ramos (U23252961)

Grupo:

8

Docente:

Milla Flores Jose Luis

Periodo:

Ciclo 7 Marzo

San Juan de Lurigancho - 2026

Introducción

El presente proyecto, denominado Urban Flow – Sistema de Gestión de Pedidos e Inventario de Ropa, tiene como finalidad desarrollar una aplicación basada en tecnologías modernas de desarrollo de software que permita optimizar la administración de productos, pedidos e inventario dentro de una tienda de ropa. El sistema fue desarrollado utilizando Spring Boot, arquitectura REST y herramientas colaborativas de desarrollo como GitHub, Git y Swagger/OpenAPI.

Durante el desarrollo del proyecto se aplicaron metodologías ágiles mediante el uso de tableros Kanban, Issues y Pull Requests en GitHub Projects, permitiendo una adecuada organización de tareas, control de avances y trabajo colaborativo entre los integrantes del equipo. Asimismo, se implementaron módulos REST para la gestión de productos y pedidos, facilitando operaciones CRUD de manera estructurada y eficiente.

Como parte de las buenas prácticas de desarrollo, también se integró documentación automática de la API mediante Swagger/OpenAPI, permitiendo visualizar y probar los endpoints desde una interfaz interactiva. Además, se realizó la simulación y resolución de conflictos de versiones utilizando Git y Visual Studio Code, fortaleciendo la experiencia en control de versiones y trabajo en equipo.

Finalmente, este proyecto busca reforzar las competencias relacionadas con el desarrollo backend, el uso de herramientas colaborativas y la implementación de flujos de trabajo profesionales en entornos de desarrollo de software, integrando conocimientos técnicos y metodologías ágiles para la construcción de soluciones tecnológicas eficientes.

División Completa del Hito 2 (Semana 10 - APF2)

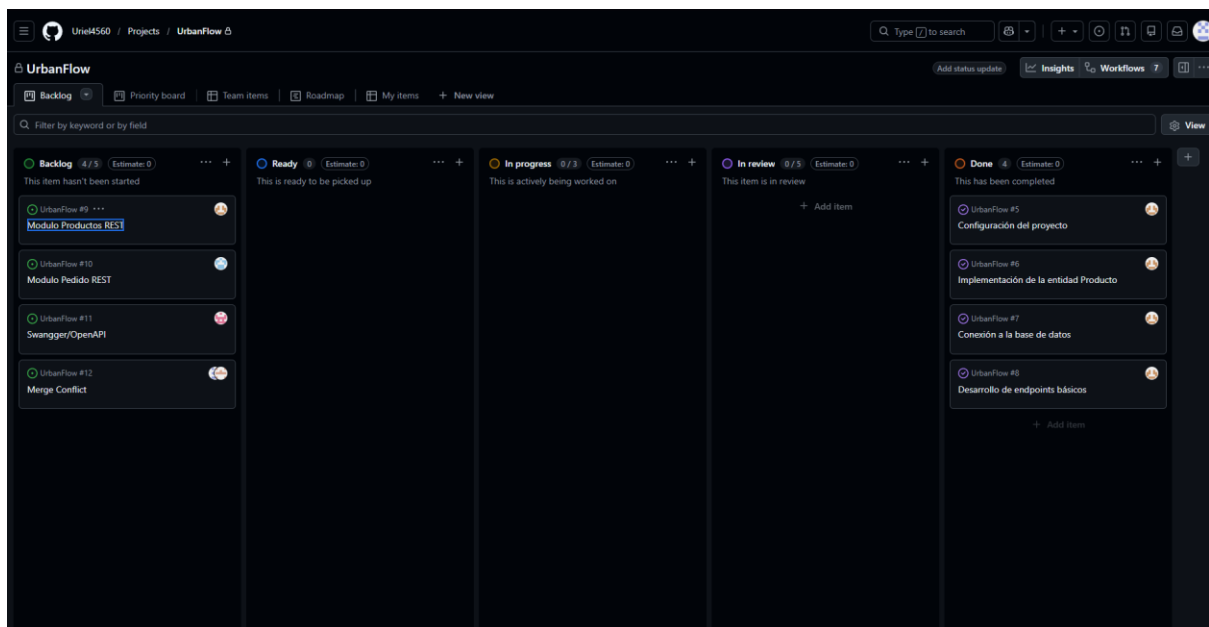
Gestión Ágil (Kanban + Issues)

En el desarrollo del proyecto se implementó una metodología ágil utilizando las herramientas de GitHub, específicamente GitHub Projects y GitHub Issues.

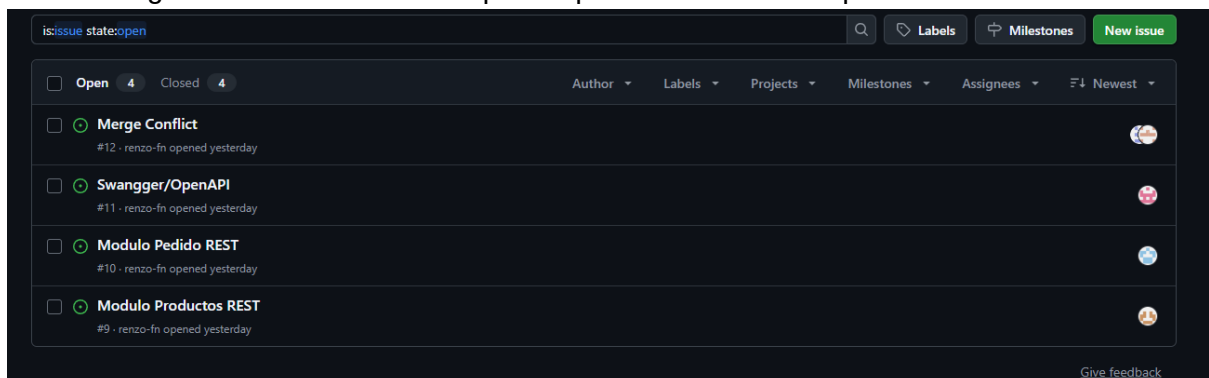
El objetivo fue organizar adecuadamente las tareas del equipo, realizar seguimiento al progreso del proyecto y facilitar el trabajo colaborativo mediante un tablero Kanban.

Responsabilidades

- Configurar el tablero en GitHub Projects. Se creó un tablero dentro del repositorio del proyecto utilizando GitHub Projects



- Crear Issues para cada tarea
Se generaron distintos Issue para representar las tareas pendientes del sistema.



- Asignar responsables
Cada tarea fue asignada a un integrante del equipo mediante la opción de Assignees de GitHub. Esto permitió establecer responsabilidades claras y mejorar la coordinación entre los miembros del grupo.

- Organizar columnas:

Backlog

Contiene tareas pendientes de planificación o priorización.

To Do

Incluye actividades listas para ser desarrolladas.

In Progress

Representa tareas que se encuentran actualmente en desarrollo.

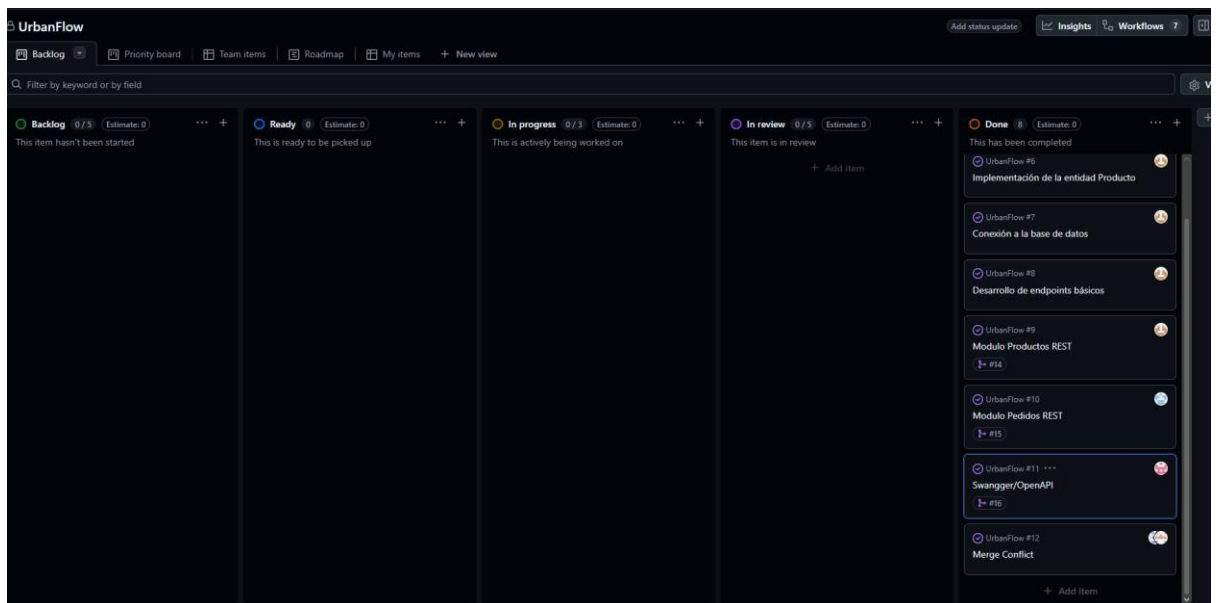
Review

Incluye tareas terminadas pendientes de revisión o aprobación mediante Pull Request.

Done

Contiene tareas completamente finalizadas.

- Verificar que cada Pull Request cierre automáticamente un Issue usando:
Closes #9-12
- Evidencia de tareas en Done



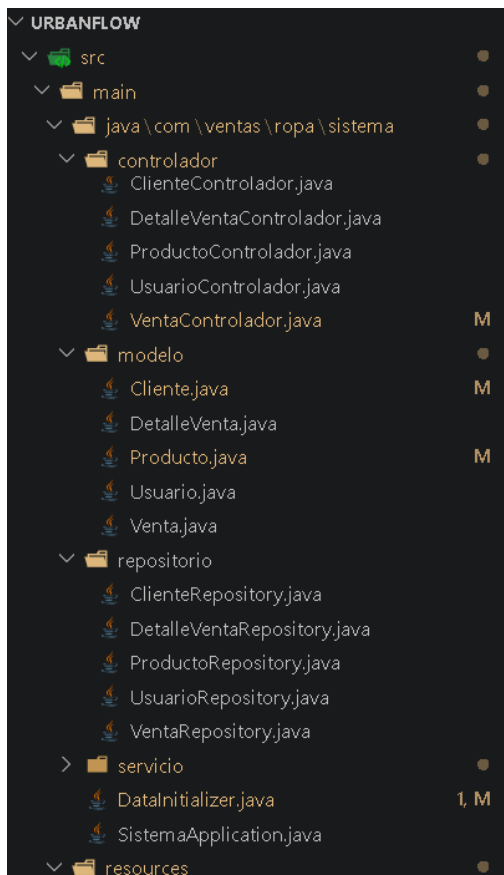
Módulo Productos REST

En el desarrollo del proyecto, se implementó el módulo REST de productos utilizando SpringBoot. Este módulo permite gestionar operaciones CRUD(Crear, Leer, Actualizar y Eliminar).

Responsabilidades

Desarrollar el módulo completo de Productos dentro de src/.

- En primer lugar, se crearon primero los archivos del módulo REST de productos en vsc.



- Después de realizar el desarrollo del módulo, utilizamos git bash para poder crear la rama feature/productos donde se subirá el archivo y los cambios realizados.

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (feature/productos)
$ git branch
  develop
  feature/README
  feature/bd
* feature/productos
  feature/springboot
  main
```

- Se escribió el comando git status para verificar los archivos que se han modificado.

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (feature/productos)
$ git status
On branch feature/productos
Your branch is up to date with 'origin/feature/productos'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    create-urbanflow-db.sql
        modified:   frontend/app-admin.js
        modified:   frontend/app-ecommerce.js
        modified:   frontend/ecommerce.html
        modified:   frontend/styles-ecommerce.css
        modified:   frontend/styles-login.css
        modified:   src/main/java/com/ventas/ropa/sistema/DataInitializer.java
        modified:   src/main/java/com/ventas/ropa/sistema/controlador/VentaControlador.java
        modified:   src/main/java/com/ventas/ropa/sistema/modelo/Cliente.java
        modified:   src/main/java/com/ventas/ropa/sistema/modelo/Producto.java
        modified:   src/main/java/com/ventas/ropa/sistema/servicio/UsuarioServicio.java
        modified:   src/main/resources/application.properties
        modified:   src/main/resources/static/app-ecommerce.js
        modified:   src/main/resources/static/ecommerce.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        informes/informe_modulo_productos.md
        query
        src/test/java/com/ventas/ropa/sistema/DataInitializerTest.java
        src/test/java/com/ventas/ropa/sistema/servicio/
        urbanflow.sql
```

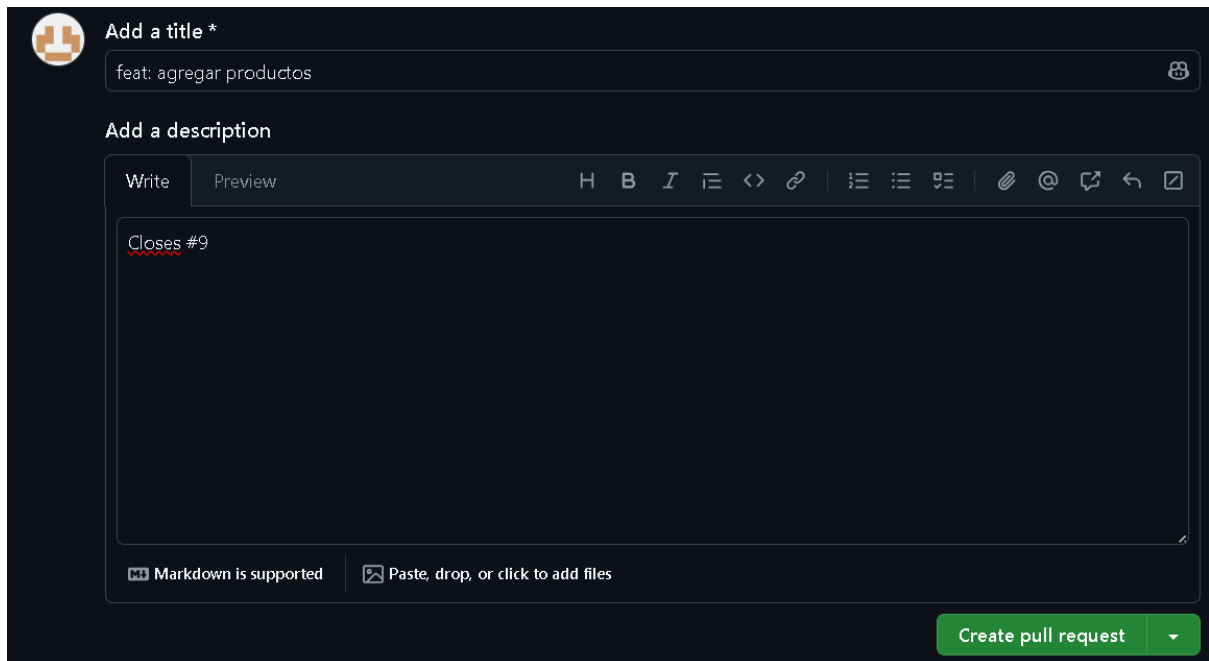
- Se uso git add . para agregar los cambios realizados , luego git commit -m para avisar el cambio realizado y por último git push para subir

```
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (feature/productos)
$ git add .

leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (feature/productos)
$ git commit -m "Feat: agregar productos"
[feature/productos 92fc376] feat: agregar productos
18 files changed, 748 insertions(+), 256 deletions(-)
create mode 100644 informes/informe_modulo_productos.md
create mode 100644 query
create mode 100644 src/test/java/com/ventas/ropa/sistema/DataInitializerTest.java
create mode 100644 src/test/java/com/ventas/ropa/sistema/servicio/UsuarioServicioTest.java
rename create-urbanflow-db.sql => urbanflow.sql (94%)

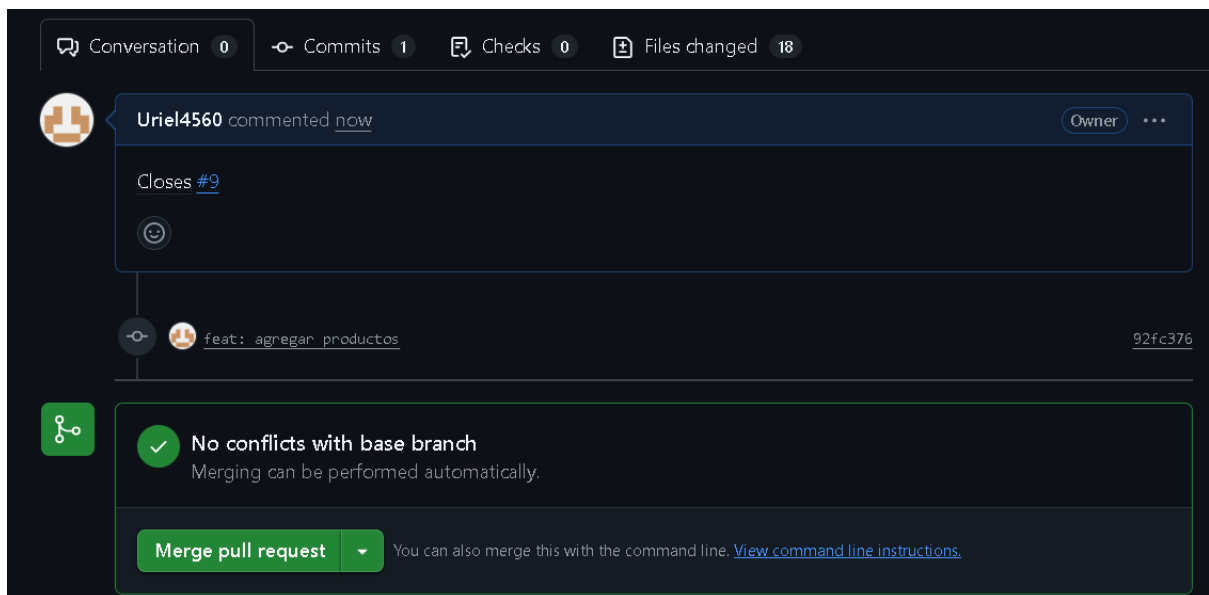
leo45@Uriel MINGW64 ~/OneDrive/Escritorio/UrbanFlow (feature/productos)
$ git push
Enumerating objects: 70, done.
Counting objects: 100% (70/70), done.
Delta compression using up to 16 threads
Compressing objects: 100% (31/31), done.
Writing objects: 100% (41/41), 12.83 KiB | 3.21 MiB/s, done.
Total 41 (delta 18), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (18/18), completed with 15 local objects.
To https://github.com/Uriel4560/UrbanFlow.git
def005a..92fc376 feature/productos -> feature/productos
```

- En el repositorio GitHub aplicamos el pull request , en su descripcion escribimos Closes #9



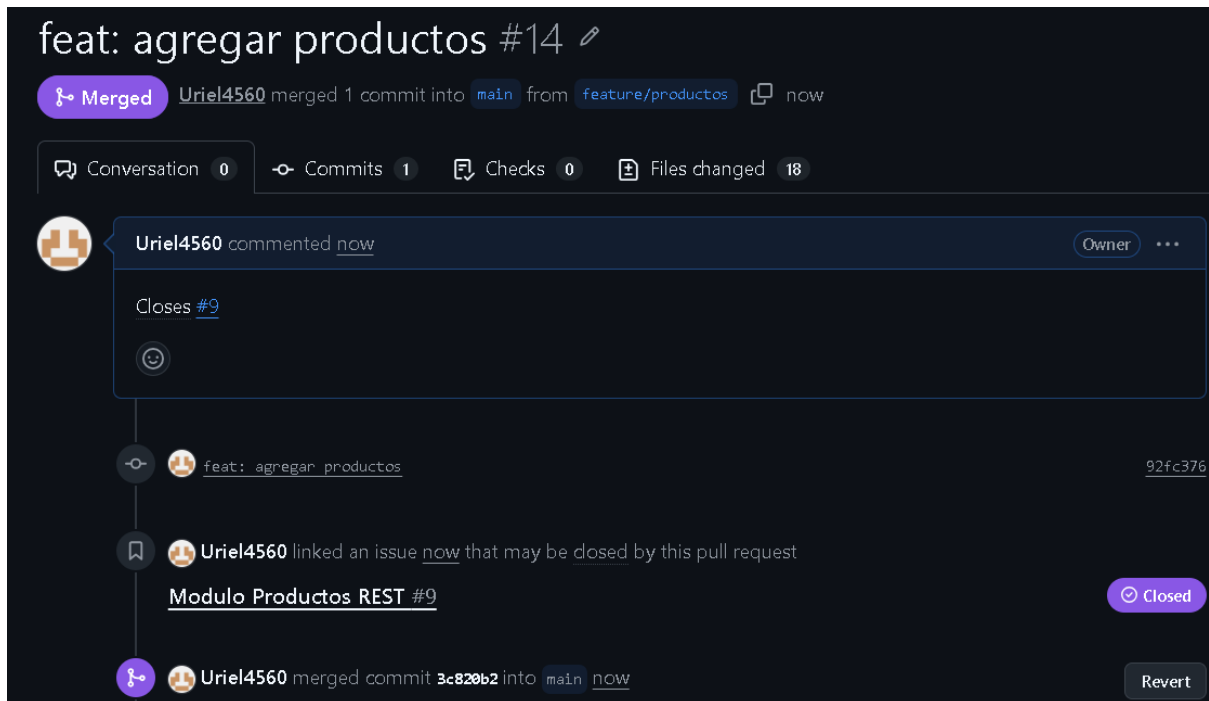
The screenshot shows the GitHub interface for creating a pull request. At the top, there's a header with a profile icon and the text 'Add a title *'. Below this is a text input field containing 'feat: agregar productos'. To the right of the input is a lock icon. Below the title field is a section titled 'Add a description'. It contains a 'Write' tab and a 'Preview' tab. The 'Write' tab is active, showing a rich text editor with the text 'Closes #9'. The editor has a toolbar with various formatting options like bold, italic, link, and list. Below the editor, there are two buttons: 'Markdown is supported' and 'Paste, drop, or click to add files'. At the bottom right, there is a green button labeled 'Create pull request' with a dropdown arrow.

- Luego de ellos, se realizó un merge para fusionar la rama feature/productos con el main.

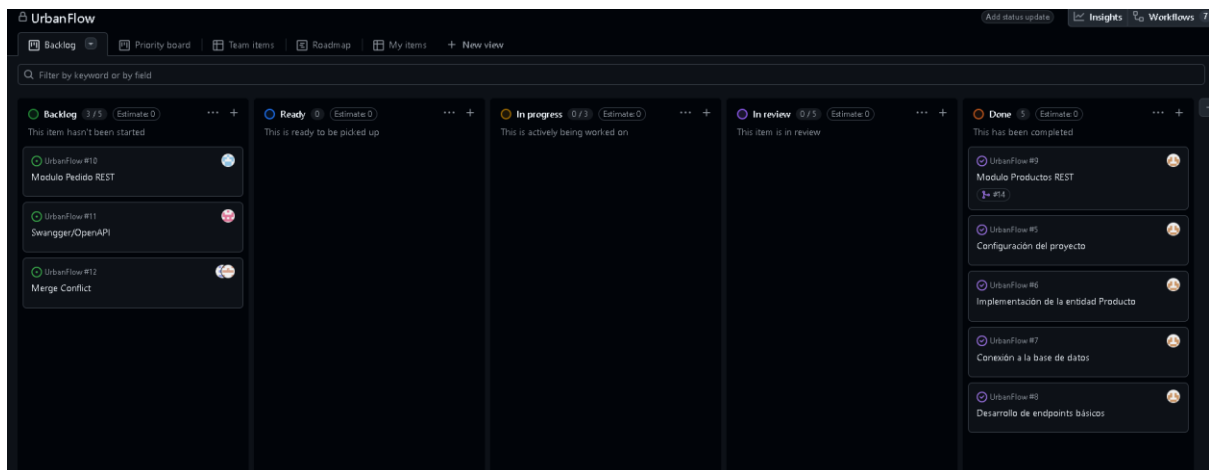


The screenshot shows the GitHub pull request page. At the top, there's a navigation bar with tabs: 'Conversation' (0), 'Commits' (1), 'Checks' (0), and 'Files changed' (18). Below the navigation bar is a comment by 'Uriel4560' with the text 'Closes #9'. Below the comment is a commit by 'Uriel4560' with the message 'feat: agregar productos' and the hash '92fc376'. Below the commit is a green box with a checkmark and the text 'No conflicts with base branch'. Below this text is a green button labeled 'Merge pull request' with a dropdown arrow. To the right of the button is a link: 'You can also merge this with the command line. [View command line instructions.](#)'

- Se muestra que se realizó con éxito el merge pull request



- Por último, en el tablero kanban se muestra que la issue se encuentra en Done, lo que significa que la tarea ya se realizó.



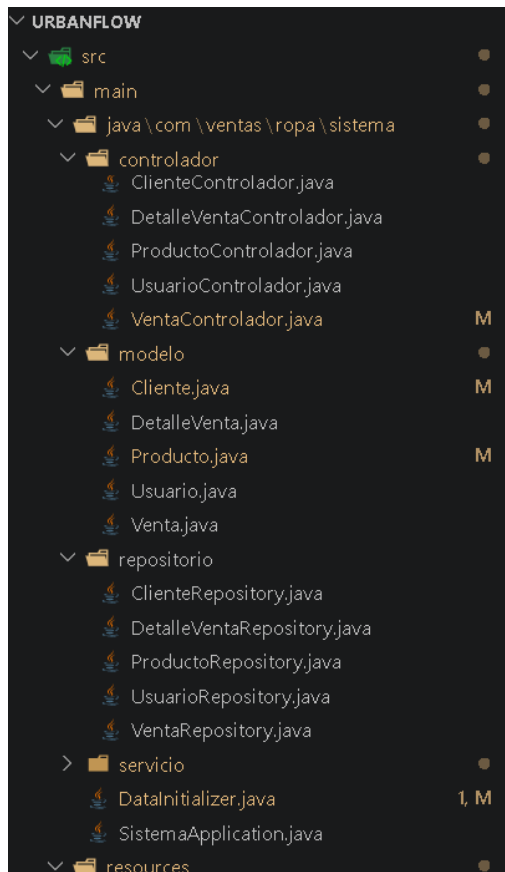
Módulo Ventas(pedidos) REST

En el desarrollo del proyecto, se implementó el módulo REST de ventas(pedidos) utilizando Spring Boot. Este módulo permite gestionar operaciones CRUD(Crear, Leer, Actualizar y Eliminar), aplicando arquitectura REST y organización de capas.

Responsabilidades

Desarrollar el módulo completo de ventas(pedidos) dentro de src/.

- En primer lugar, se crearon primero los archivos del módulo REST de productos en vscode.



- Después de realizar el desarrollo del módulo, utilizamos git bash para poder crear la rama feature/pedidos donde se subirá el archivo y los cambios realizados.

```
ChristianPC@DESKTOP-L5J3RE2 MINGW64 ~/Desktop/D/UrbanFlow (feature/pedidos)
$ git branch
* feature/pedidos
main
```

- Se utilizó el comando git checkout feature/pedidos para integrarnos en la rama indicada.

```
ChristianPC@DESKTOP-L5J3RE2 MINGW64 ~/Desktop/D/UrbanFlow (main)
$ git checkout feature/pedidos
branch 'feature/pedidos' set up to track 'origin/feature/pedidos'.
Switched to a new branch 'feature/pedidos'
```

- Se escribió el comando git status para verificar los archivos que se han modificado.

```
ChristianPC@DESKTOP-L5J3RE2 MINGW64 ~/Desktop/D/UrbanFlow (feature/pedidos)
$ git status
On branch feature/pedidos
Your branch is up to date with 'origin/feature/pedidos'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/main/java/com/ventas/ropa/sistema/modelo/Venta.java

no changes added to commit (use "git add" and/or "git commit -a")
```

- Se usó git add . para agregar los cambios realizados.

```
ChristianPC@DESKTOP-L5J3RE2 MINGW64 ~/Desktop/D/UrbanFlow (feature/pedidos)
$ git add .
```

- Después se utilizó git commit -m para avisar el cambio realizado.

```
ChristianPC@DESKTOP-L5J3RE2 MINGW64 ~/Desktop/D/UrbanFlow (feature/pedidos)
$ git commit -m "feat: Agregar pedido de venta"
[feature/pedidos 0f22cfb] feat: Agregar pedido de venta
1 file changed, 4 insertions(+)
```

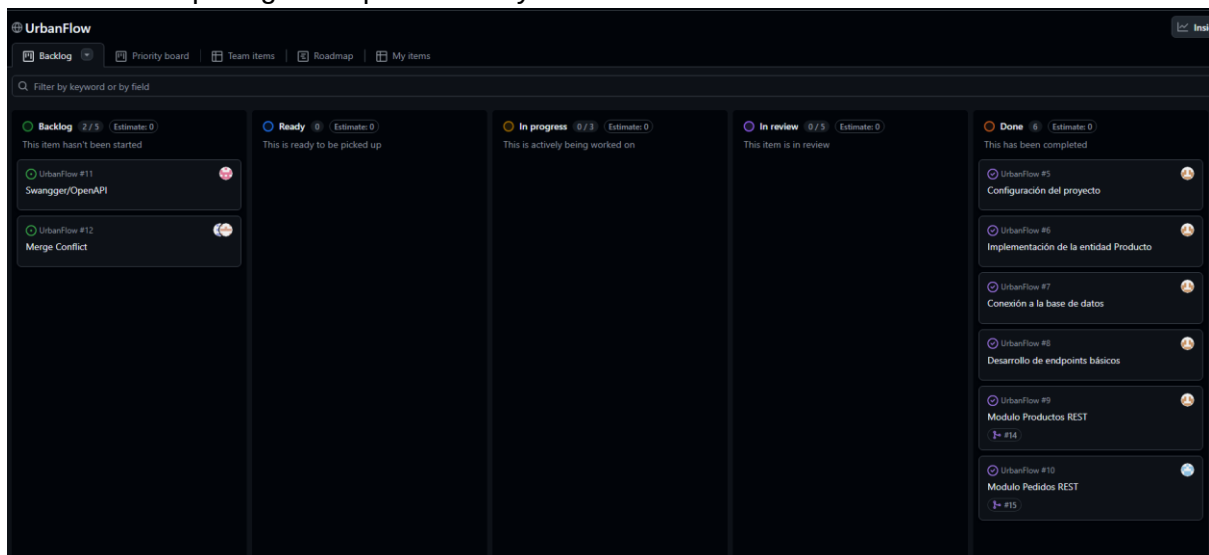
- Por último git push para subir los archivos nuevos agregados.

```
ChristianPC@DESKTOP-L5J3RE2 MINGW64 ~/Desktop/D/UrbanFlow (feature/pedidos)
$ git push
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (11/11), 782 bytes | 782.00 KiB/s, done.
Total 11 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/Uriel4560/UrbanFlow.git
   def005a..0f22cfb  feature/pedidos -> feature/pedidos
```

- En el repositorio GitHub aplicamos el pull request , en su descripción escribimos Closes #10



- Luego de ellos, se realizó un merge para fusionar la rama feature/pedidos con el main.
- Se muestra que se realizó con éxito el merge pull request
- Por último, en el tablero kanban se muestra que la issue se encuentra en Done, lo que significa que la tarea ya se realizó.



Implementación de Swagger para la documentación

Como parte del desarrollo del proyecto se integró documentación automática de la API REST utilizando Swagger en Spring Boot. Esta integración permitirá visualizar y probar los endpoints del sistema mediante una interfaz web interactiva, facilitando el proceso de documentación, pruebas y validación de la API

¿Qué es el Swagger?

Swagger es una herramienta que genera automáticamente una página web de documentación para tu API REST.

En vez de explicar manualmente cada endpoint, Swagger va a leer el código Spring Boot y creará una interfaz visual donde puedes ver todos tus endpoints (GET, POST, PUT, DELETE), probarlos directamente desde el navegador, y ver los modelos de datos (esquemas).

Responsabilidades

Desarrollar la documentación automática API usando Swagger en Spring Boot.

- En primer lugar, se agregó la dependencia maven de springdoc-openapi en el archivo pom.xml del proyecto.



```
32      <dependencies> Add Spring Boot Starters...
33          <dependency>
34              <groupId>org.springframework.boot</groupId>
35              <artifactId>spring-boot-starter-web</artifactId>
36          </dependency>
37
38          <dependency>
39              <groupId>org.springframework.boot</groupId>
40              <artifactId>spring-boot-starter-data-jpa</artifactId>
41          </dependency>
42
43          <dependency>
44              <groupId>com.mysql</groupId>
45              <artifactId>mysql-connector-j</artifactId>
46              <scope>runtime</scope>
47          </dependency>
48
49          <dependency>
50              <groupId>com.h2database</groupId>
51              <artifactId>h2</artifactId>
52              <scope>runtime</scope>
53          </dependency>
54
55          <dependency>
56              <groupId>org.springframework.boot</groupId>
57              <artifactId>spring-boot-devtools</artifactId>
58              <scope>runtime</scope>
59              <optional>true</optional>
60          </dependency>
61      </dependencies>
```

```

62     <dependency>
63       <groupId>org.springframework.security</groupId>
64       <artifactId>spring-security-crypto</artifactId>
65     </dependency>
66
67     <dependency>
68       <groupId>org.springframework.boot</groupId>
69       <artifactId>spring-boot-starter-test</artifactId>
70       <scope>test</scope>
71     </dependency>
72
73     <dependency>
74       <groupId>org.springdoc</groupId>
75       <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
76       <version>2.5.0</version>
77     </dependency>
78   </dependencies>

```

- Se levantó el servidor Spring Boot y se accedió a la interfaz Swagger UI desde el navegador para verificar que los endpoints de los módulos Productos y Pedidos (REST) aparecieran correctamente documentados.

URL de Swagger : <http://localhost:8080/swagger-ui/index.html>

The screenshot shows the Swagger UI interface for a REST API. The browser address bar displays `http://localhost:8080/swagger-ui/index.html#`. The Swagger logo is visible in the top left corner, and the search bar contains `/api-docs`. The main heading is "OpenAPI definition" with a version indicator "v0 OAS 3.0". Below this, the "Servers" section shows the default server URL: `http://localhost:8080 - Generated server url`.

The API is organized into three main sections:

- Clientes** (Gestión de clientes): This section contains five endpoints:
 - GET** `/api/clientes/{id}`: Obtener cliente por ID
 - PUT** `/api/clientes/{id}`: Actualizar cliente
 - DELETE** `/api/clientes/{id}`: Eliminar cliente
 - GET** `/api/clientes`: Obtener todos los clientes
 - POST** `/api/clientes`: Crear nuevo cliente
- Autenticación** (Endpoints de login y registro): This section contains three endpoints:
 - POST** `/api/auth/login`: Login de usuario administrativo
 - POST** `/api/auth/cliente/registro`: Registro de nuevo cliente
 - POST** `/api/auth/cliente/login`: Login de cliente
- Detalles de Venta** (Gestión de detalles de ventas): This section contains five endpoints:
 - GET** `/api/detalles-venta/{id}`: Obtener detalle de venta por ID
 - PUT** `/api/detalles-venta/{id}`: Actualizar detalle de venta
 - DELETE** `/api/detalles-venta/{id}`: Eliminar detalle de venta
 - GET** `/api/detalles-venta`: Obtener todos los detalles de venta
 - POST** `/api/detalles-venta`: Crear detalle de venta

Productos Gestión de productos del catálogo			^
GET	/api/productos/{id}	Obtener producto por ID	▼
PUT	/api/productos/{id}	Actualizar producto	▼
DELETE	/api/productos/{id}	Eliminar producto	▼
GET	/api/productos	Obtener todos los productos	▼
POST	/api/productos	Crear nuevo producto	▼
Ventas Gestión de ventas y checkout			^
GET	/api/ventas/{id}	Obtener venta por ID	▼
PUT	/api/ventas/{id}	Actualizar venta	▼
DELETE	/api/ventas/{id}	Eliminar venta	▼
GET	/api/ventas	Obtener todas las ventas	▼
POST	/api/ventas	Crear venta	▼
POST	/api/ventas/checkout	Procesar checkout	▼

- Se verificó que los schemas (modelos de datos) de ProductoDTO y PedidoDTO se mostraran correctamente en la sección Schemas de Swagger UI.

Schemas		^
Cliente	>	
Venta	>	
Producto	>	
DetalleVenta	>	

```

Cliente ▾ {
  id                > [...]
  nombre            > [...]
  apellido          > [...]
  email             > [...]
  contraseña        > [...]
  telefono          > [...]
  cedula            > [...]
  ciudad            > [...]
  direccion         > [...]
  activo            > [...]
  fechaCreacion     > [...]
  fechaActualizacion > [...]
}

```

```
Venta ▾ {
  id > [...]
  numeroVenta > [...]
  cliente Cliente > {...}
  total > [...]
  estado > [...]
  metodoPago > [...]
  observaciones > [...]
  fechaVenta > [...]
  fechaEntrega > [...]
}
```

```
Producto ▾ {
  id > [...]
  codigo > [...]
  nombre > [...]
  descripcion > [...]
  categoria > [...]
  marca > [...]
  talla > [...]
  color > [...]
  precio > [...]
  stock > [...]
  stockMinimo > [...]
  urlImagen > [...]
  material > [...]
  activo > [...]
  fechaCreacion > [...]
  fechaActualizacion > [...]
}
```

```
DetalleVenta ▾ {
  id > [...]
  venta Venta > {...}
  producto Producto > {...}
  cantidad > [...]
  precioUnitario > [...]
  subtotal > [...]
}
```

- Se creó la rama feature/swagger desde develop utilizando el comando:
git checkout -b feature/swagger

```
Administrator@Jair MINGW64 ~/Documents/UrbanFlow/UrbanFlow (main)
$ git checkout -b feature/swagger
Switched to a new branch 'feature/swagger'
```

- Se escribió el comando git status para verificar los archivos que se han modificado.

```
Administrator@Jair MINGW64 ~/Documents/UrbanFlow/UrbanFlow (feature/swagger)
$ git status
On branch feature/swagger
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   pom.xml
    modified:   src/main/java/com/ventas/ropa/sistema/controlador/ClienteCon
trolador.java
    modified:   src/main/java/com/ventas/ropa/sistema/controlador/DetalleVen
taControlador.java
    modified:   src/main/java/com/ventas/ropa/sistema/controlador/ProductoCo
ntrolador.java
    modified:   src/main/java/com/ventas/ropa/sistema/controlador/UsuarioCon
trolador.java
    modified:   src/main/java/com/ventas/ropa/sistema/controlador/VentaContr
olador.java
    modified:   src/main/resources/application.properties

no changes added to commit (use "git add" and/or "git commit -a")
```

- Se utilizaron los comandos git add . , git commit -m "feat: integrar swagger/openapi" y git push origin feature/swagger para subir los cambios al repositorio remoto.

```
Administrator@Jair MINGW64 ~/Documents/UrbanFlow/UrbanFlow (feature/swagger)
$ git add .

Administrator@Jair MINGW64 ~/Documents/UrbanFlow/UrbanFlow (feature/swagger)
$ git commit -m "feat:agregar swagger para la documentacion de endpoints"
[feature/swagger 6706f06] feat:agregar swagger para la documentacion de endpoi
nts
7 files changed, 213 insertions(+), 52 deletions(-)
```

```
Administrator@Jair MINGW64 ~/Documents/UrbanFlow/UrbanFlow (feature/swagger)
$ git push -u origin feature/swagger
info: please complete authentication in your browser...
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 16 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (18/18), 6.65 KiB | 6.65 MiB/s, done.
Total 18 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 4 local objects.
To https://github.com/Uriel4560/UrbanFlow.git
   def005a..6706f06  feature/swagger -> feature/swagger
branch 'feature/swagger' set up to track 'origin/feature/swagger'.
```


- En el repositorio GitHub se aplicó el Pull Request con la descripción Closes #11.

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about d](#)

base: `main` ← compare: `feature/swagger` ✓ **Able to merge.** These branches can be automatically merged.

 Add a title *

feat:agregar swagger para la documentacion de endpoints

Add a description

Write Preview

Closes #11

Markdown is supported Paste, drop, or click to add files

Create pull request

- Se realizó en feature/swagger el merge pull request con la rama main.

feat:agregar swagger para la documentacion de endpoints #16

 Open jair-md wants to merge 1 commit into `main` from `feature/swagger`

Conversation 0 Commits 1 Checks 0 Files changed 7

 jair-md commented now Collaborator

Closes #11

 feat:agregar swagger para la documentacion de endpoints 6706f06

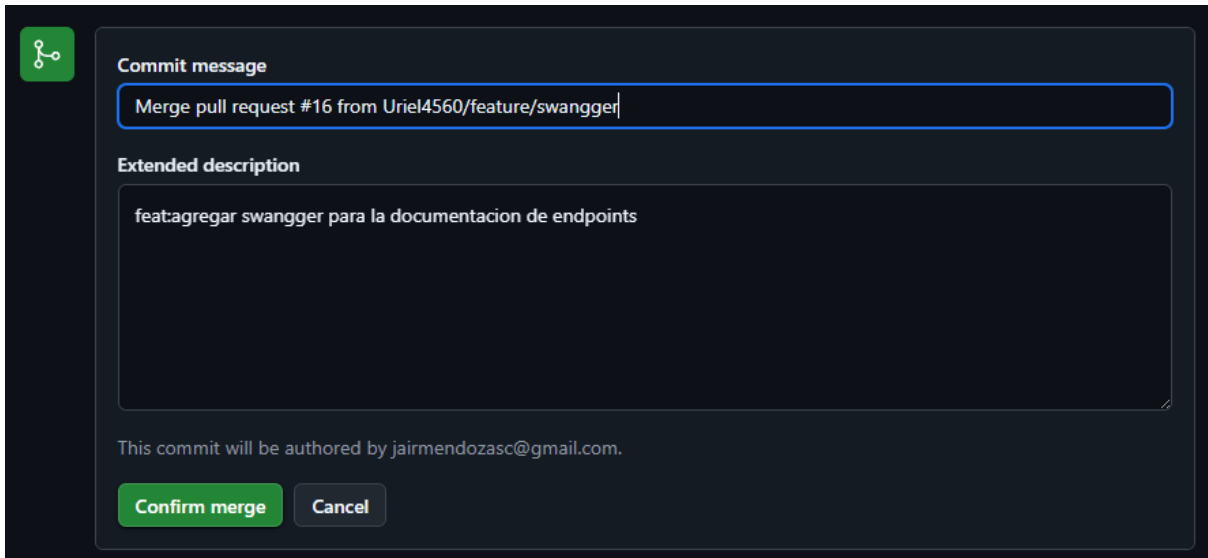
 **No conflicts with base branch**
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Reviewers: Suggested: Uriel
Still in progress: Assignees: No one
Labels: None yet
Projects: None yet
Milestones: No milestones

- Se visualiza el commit del merge pull request para luego seleccionar en confirmar merge

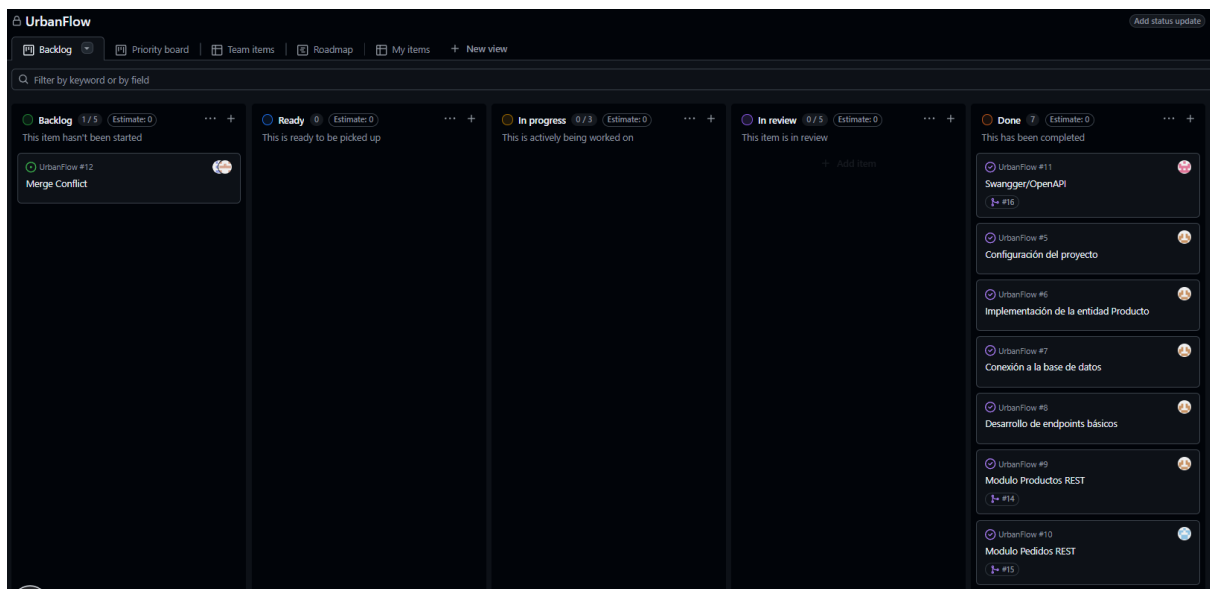


The screenshot shows the 'Commit message' dialog in GitHub. The 'Commit message' field contains the text 'Merge pull request #16 from Uriel4560/feature/swangger'. Below it, the 'Extended description' field contains the text 'feat:agregar swangger para la documentacion de endpoints'. At the bottom, it states 'This commit will be authored by jairmendozasc@gmail.com.' and has two buttons: 'Confirm merge' (green) and 'Cancel' (grey).

- Se muestra que se realizo con exito el merge pull request



The screenshot shows the GitHub pull request page for 'feat:agregar swangger para la documentacion de endpoints #16'. The status is 'Merged'. The merge message says 'jair-md merged 1 commit into main from feature/swangger now'. Below this, there are statistics: 'Conversation 0', 'Commits 1', 'Checks 0', and 'Files changed 7'. A comment from 'jair-md' 1 minute ago says 'Closes #11'. Below the comment, there is a commit 'feat:agregar swangger para la documentacion de endpoints' with hash '6706f06'. Another comment from 'jair-md' 1 minute ago says 'Linked an issue 1 minute ago that may be closed by this pull request' and points to 'Swagger/OpenAPI #11'. Below that, a comment from 'jair-md' says 'merged commit 411ca5f into main now'. At the bottom, there is a message 'Pull request successfully merged and closed' with a 'Delete branch' button. Below this is a section 'Add a comment' with a text area and a 'Comment' button.



- Por último, se visualiza en el kanban la tarea como Done, confirmando que esta se realizó y concluyó.

Merge Conflict + Documentación

Durante el desarrollo colaborativo del proyecto se realizó una simulación de conflicto de versiones utilizando GitHub y Git.

El objetivo de esta actividad fue provocar intencionalmente un Merge Conflict, documentar el problema y aplicar correctamente el proceso de resolución utilizando Visual Studio Code.

Responsabilidades

Provocar y resolver un conflicto Git documentado, simulando una situación real donde dos desarrolladores modifican el mismo archivo al mismo tiempo .

Archivo requerido

Se creó el siguiente archivo para documentar el proceso realizado:
documentos/hito2/04_RESOLUCION_CONFLICTOS.md

Debe incluir

1. Archivos en conflicto

Archivo	Rama A (feature/conflicto-A)	Rama B (feature/conflicto-B)
config.txt	version=2.0, puerto=9090, base_datos=urbanflow_dev	version=3.0, puerto=7070, base_datos=urbanflow_test

Representa una configuración hipotética del sistema. Dos desarrolladores trabajaron en paralelo y ambos modificaron las mismas líneas sin coordinarse, generando un conflicto.

2. Explicación del conflicto

- Qué cambios chocaron
Ambas ramas modificaron las mismas 3 líneas del archivo config.txt:
 - Línea 1: version=2.0 vs version=3.0
 - Línea 2: puerto=9090 vs puerto=7070
 - Línea 3: base_datos=urbanflow_dev vs base_datos=urbanflow_test
- Qué ramas participaron
 - feature/conflicto-A (creada desde develop)
 - feature/conflicto-B (creada desde develop)
 - feature/conflictos (rama que recibió el merge y resolvió el conflicto)

Esto ocurre cuando dos desarrolladores trabajan en paralelo desde develop, modifican el mismo archivo y luego intentan fusionar sus cambios. Git no puede decidir automáticamente qué versión es la correcta

3. Resolución en VS Code

- Uso de Source Control
 - Se abrió el archivo config.txt en VS Code
 - VS Code detectó automáticamente el conflicto y marcó las líneas en conflicto con colores rojo/verde
 - Se abrió la vista de Source Control (Ctrl+Shift+G) para visualizar los archivos en conflicto
 - Se utilizó la opción "Accept Both Changes" para combinar ambas versiones
- Accept Both Changes

```
versión=2.0
puerto=9090
base_datos=urbanflow_dev
versión=3.0
puerto=7070
base_datos=urbanflow_test
```

- Decisión tomada
Se eligió "Accept Both Changes" porque ambas versiones aportaban valores útiles:
 - La versión 3.0 era más reciente
 - El puerto 9090 era el que ya funcionaba en pruebas anteriores
 - La base de datos urbanflow_test era la correcta para el entorno de pruebas

4. Comandos Git usados

Crear rama de trabajo

```
git checkout develop
```

```
git checkout -b feature/conflictos
```

Fusionar conflicto-A (sin conflicto)

git merge feature/conflicto-A

Fusionar conflicto-B (genera CONFLICTO)

git merge feature/conflicto-B

Resolver conflicto manualmente

git add config.txt

git commit -m "merge: resolver conflicto entre conflicto-A y conflicto-B"

Subir cambios

git push origin feature/conflictos

Comandos auxiliares utilizados:

Ver todas las ramas

git branch -a

Ver historial de commits

git log --oneline -10

Ver estado actual

git status

Rama

feature/conflictos

Evidencia

- Terminal con mensaje de conflicto git merge:

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git commit -m "base: crear config.txt para prueba de conflictos"
On branch feature/resolver-conflicto-doc
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  config.txt

nothing added to commit but untracked files present (use "git add" to track)

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the ne
xt time Git touches it

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git commit -m "base: crear config.txt para prueba de conflictos"
[feature/resolver-conflicto-doc 2ba1710] base: crear config.txt para prueba de c
onflictos
1 file changed, 3 insertions(+)
create mode 100644 config.txt
```

- Archivo config.txt con marcadores <<<<<< HEAD:

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git checkout -b feature/conflicto-A
Switched to a new branch 'feature/conflicto-A'

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-A)
$ echo "version=2.0" > config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-A)
$ echo "base_datos=urbanflow_dev" >> config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-A)
$ echo "puerto=9090" >> config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-A)
$ cat config.txt
version=2.0
base_datos=urbanflow_dev
puerto=9090
```

- VS Code con Source Control y conflicto:

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-A)
$ git checkout feature/resolver-conflicto-doc
Switched to branch 'feature/resolver-conflicto-doc'

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git checkout -b feature/conflicto-B
Switched to a new branch 'feature/conflicto-B'

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ echo "version=3.0" > config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ echo "puerto=7070" >> config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ echo "base_datos=urbanflow_test" >> config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ cat config.txt
version=3.0
puerto=7070
base_datos=urbanflow_test

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the ne
xt time Git touches it

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ git commit -m "cambio-B: actualizar configuracion a v3.0"
[feature/conflicto-B 3a5b1c4] cambio-B: actualizar configuracion a v3.0
1 file changed, 3 insertions(+), 3 deletions(-)
```

- Botón "Merge pull request":

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ git checkout feature/resolver-conflicto-doc
Switched to branch 'feature/resolver-conflicto-doc'

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git merge feature/conflicto-A
Updating 2ba1710..c25d34e
Fast-forward
 config.txt | 6 +++---
 1 file changed, 3 insertions(+), 3 deletions(-)

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git merge feature/conflicto-B
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc|MERGING)
$ cat config.txt
<<<<<<< HEAD
version=2.0
base_datos=urbanflow_dev
puerto=9090
=====
version=3.0
puerto=7070
base_datos=urbanflow_test
>>>>>>> feature/conflicto-B
```

- Issue #12 cerrado automáticamente:

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc|MERGING)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc|MERGING)
$ git status
On branch feature/resolver-conflicto-doc
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   config.txt

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc|MERGING)
$ git commit -m "merge: resolver conflicto entre conflicto-A y conflicto-B, mantener version 3.0, puerto 9090, base urbanflow_test"
[feature/resolver-conflicto-doc 6251838] merge: resolver conflicto entre conflicto-A y conflicto-B, mantener version 3.0, puerto 9090, base urbanflow_test

ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/resolver-conflicto-doc)
$ git push origin feature/resolver-conflicto-doc
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 406 bytes | 406.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Uriel4560/UrbanFlow.git
 2ba1710..6251838 feature/resolver-conflicto-doc -> feature/resolver-conflicto-doc
```


- Ramas que participaron

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ git branch -a
  develop
  feature/bd
  feature/conflicto-A
* feature/conflicto-B
  feature/resolver-conflicto-doc
  feature/springboot
  main
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/feature/README
remotes/origin/feature/bd
remotes/origin/feature/conflicto-A
remotes/origin/feature/conflicto-B
remotes/origin/feature/resolver-conflicto-doc
remotes/origin/feature/springboot
remotes/origin/main
```

- Historial de commits

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ git log --oneline -10
fec1b96 (HEAD -> feature/conflicto-B) merge: resolver conflicto entre conflicto-A y conf
3a5b1c4 (origin/feature/conflicto-B) cambio-B: actualizar configuracion a v3.0
c25d34e (origin/feature/conflicto-A, feature/conflicto-A) cambio-A: actualizar configura
2ba1710 base: crear config.txt para prueba de conflictos
299bf37 (origin/develop, develop) docs: agregar documentacion Hito 2
446b469 feat: documento actualizado
72299fb feat:actualizando proyecto
4748de0 Primer commit
```

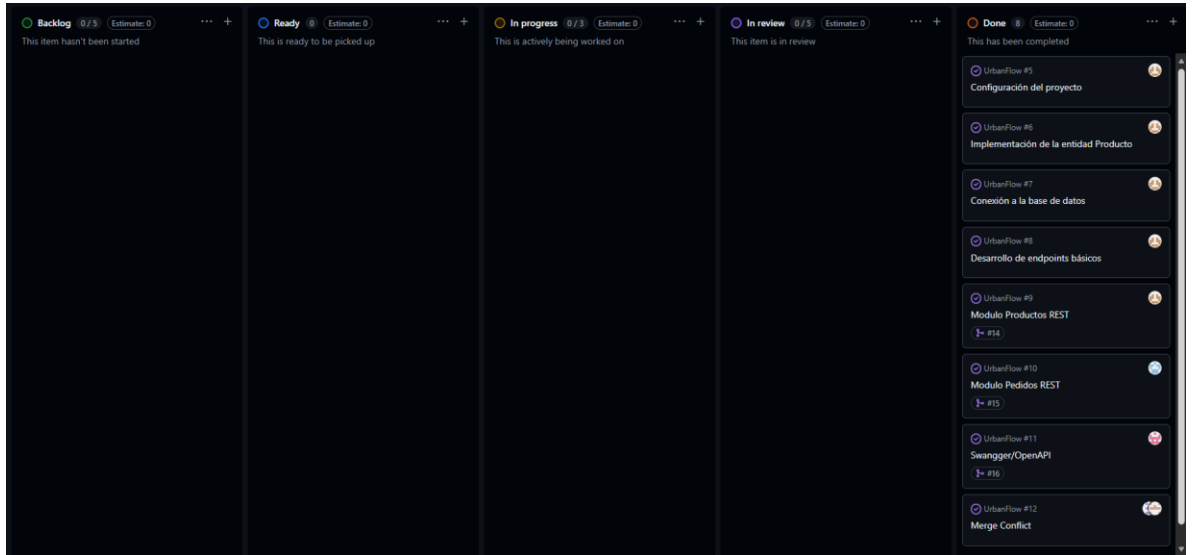
- Resolución final del conflicto:

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ cat config.txt
version=3.0
puerto=9090
base_datos=urbanflow_test
```

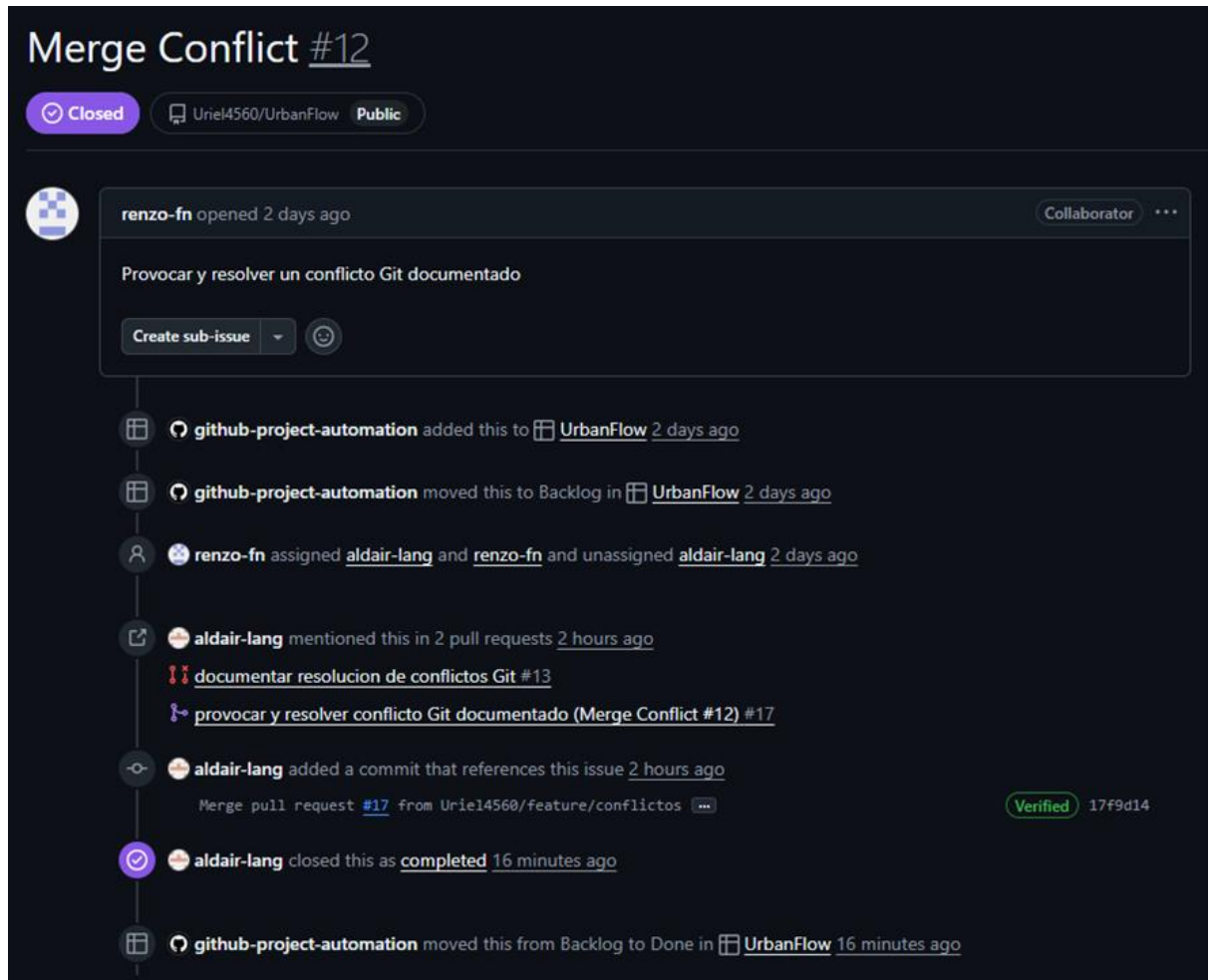
- Conflictos pendientes

```
ALDAIR@LAPTOP-CKK9M5LG MINGW64 ~/UrbanFlow (feature/conflicto-B)
$ git status
On branch feature/conflicto-B
nothing to commit, working tree clean
```


- Kanban - Tarea en Done



- Issue #12 cerrado



Github general del proyecto:

Flujo Git Recomendado

Rama principal

main -> Versión estable del proyecto (solo código probado)

Rama de integración

develop -> Integración de todas las ramas `feature`

Ramas feature

feature/productos -> Desarrollo del CRUD de productos

feature/pedidos -> Desarrollo del CRUD de pedidos

feature/swagger -> Documentación de la API con Swagger

feature/conflictos -> Prueba y resolución de conflictos Git

Recomendación Final

Estructura final del proyecto

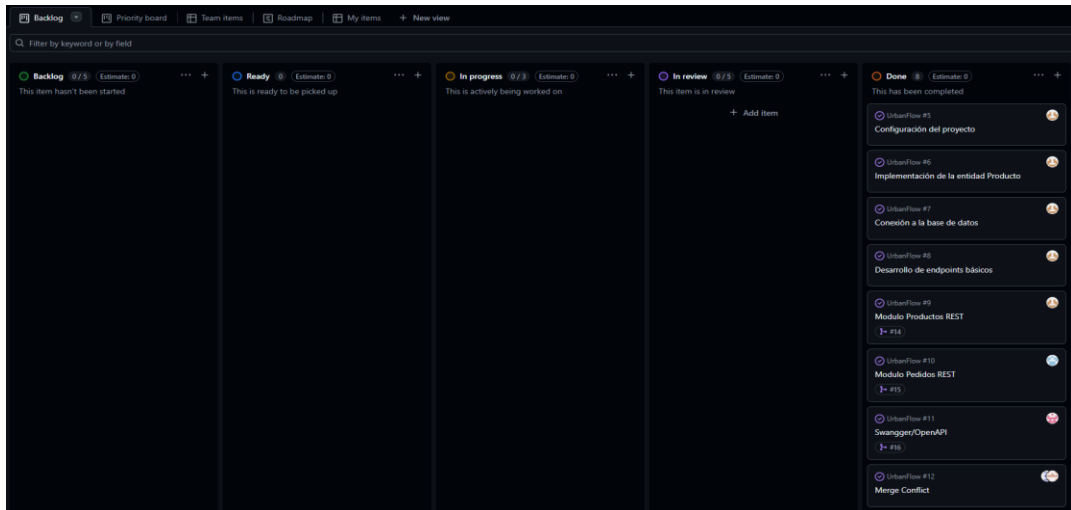
```
project/
├── src/
├── docs/
│   └── resolucion_conflicto.md
├── README.md
├── pom.xml
└── .gitignore
```

Recomendación:

- No trabajar directamente sobre `main` o `develop`
- Usar Pull Requests para fusionar ramas
- Mantener mensajes de commit claros

Evidencias que deben entregar

- GitHub Projects funcionando



- Issues vinculados

is:issue state:closed						Labels	Milestones	New issue			
☐	Open	0	Closed	8		Author	Labels	Projects	Milestones	Assignees	Sort
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐	☒										
☐											

- Merge conflict documentado

Informe Hito 2 (APF2) - UrbanFlow

Integrantes

Integrante	Rol
Christian Rivas Aquino	Diseño BD
Uriel Vásquez Quispe	Módulo Productos
Renzo Estefano Yupanqui	Gestión Ágil
David Jair Mendoza Figueroa	Swagger + Calidad
Aldair Ronaldo Casimiro Ramos	Merge + Documentación

Merge Conflict + Documentación

Integrante: Aldair Ronaldo Casimiro Ramos

Rama utilizada: `feature/conflictos`

Archivos en conflicto

Archivo	Rama <code>feature/conflicto-A</code>	Rama <code>feature/conflicto-B</code>
<code>config.txt</code>	version=2.0, puerto=9090	version=3.0, puerto=7070

Merge Conflict + Documentación

Integrante: Aldair Ronaldo Casimiro Ramos

Rama utilizada: `feature/conflictos`

Archivos en conflicto

Archivo	Rama <code>feature/conflicto-A</code>	Rama <code>feature/conflicto-B</code>
<code>config.txt</code>	version=2.0, puerto=9090	version=3.0, puerto=7070

Explicación del conflicto

¿Qué cambios chocaron?

- Línea 1: `version=2.0` vs `version=3.0`
- Línea 2: `puerto=9090` vs `puerto=7070`

¿Qué ramas participaron?

- `feature/conflicto-A`
- `feature/conflicto-B`
- `feature/conflictos`

Resolución en VS Code

Se utilizó "Accept Both Changes" y ajuste manual.

Conflicto generado

```
$ git merge feature/conflicto-B
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```



- ```
e3f0c97 (HEAD -> develop, origin/develop) docs: restaurar informe completo con seccion Merge Conflict
* 5d36ea3 (feature/conflicto-B) merge: integrar cambios remotos

17f9d14 Merge pull request #17 from Uriel4560/feature/conflictos

def005a (origin/main, origin/HEAD, main) feat:actualizando info
33f0a1b feat: actualizar documentos e informes
c169dd2 feat: actualizar bd parte 2
9a0e934 Merge pull request #4 from Uriel4560/feature/springboot

7c99a86 (origin/feature/springboot, feature/springboot) feat: springboot actualizado
bcde612 Merge pull request #3 from Uriel4560/feature/bd

f7c602c (origin/feature/bd, feature/bd) feat: base de datos actualizado

be8fcd9 Merge pull request #2 from Uriel4560/feature/README

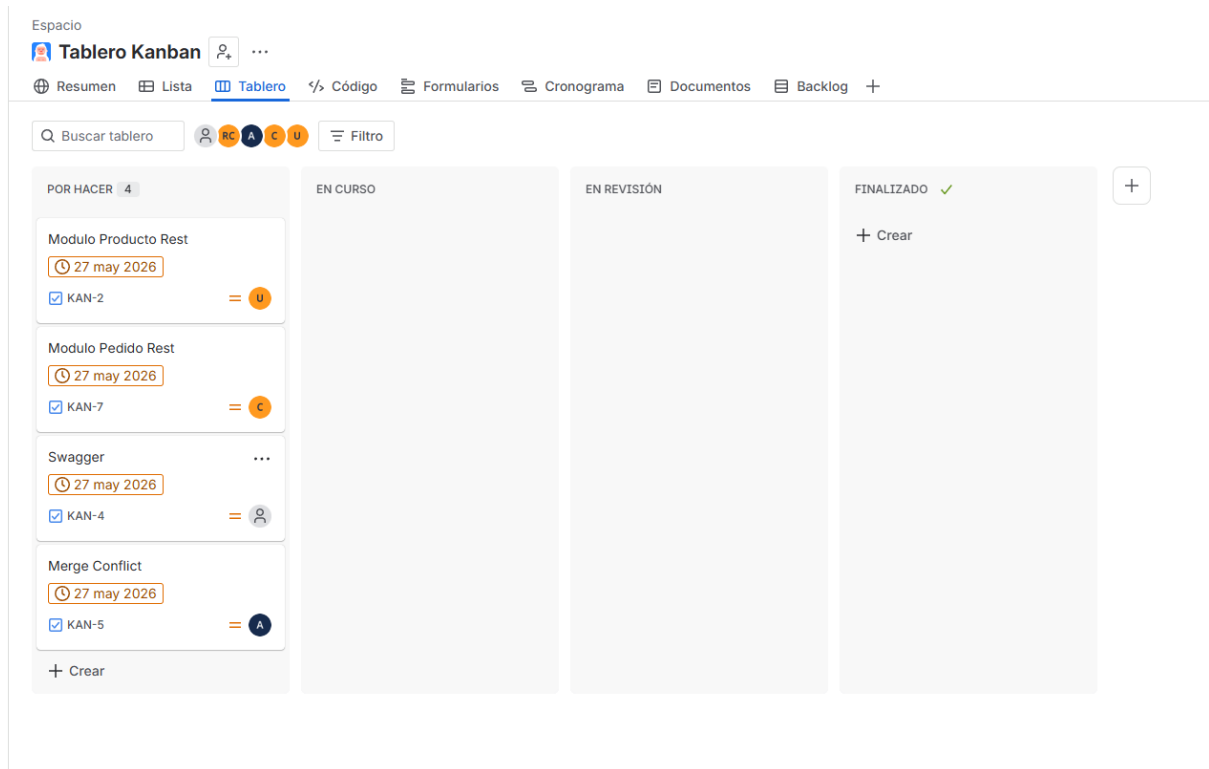
ccb97cb (origin/feature/README) feat:README actualizado

b7cad20 Merge pull request #1 from Uriel4560/develop

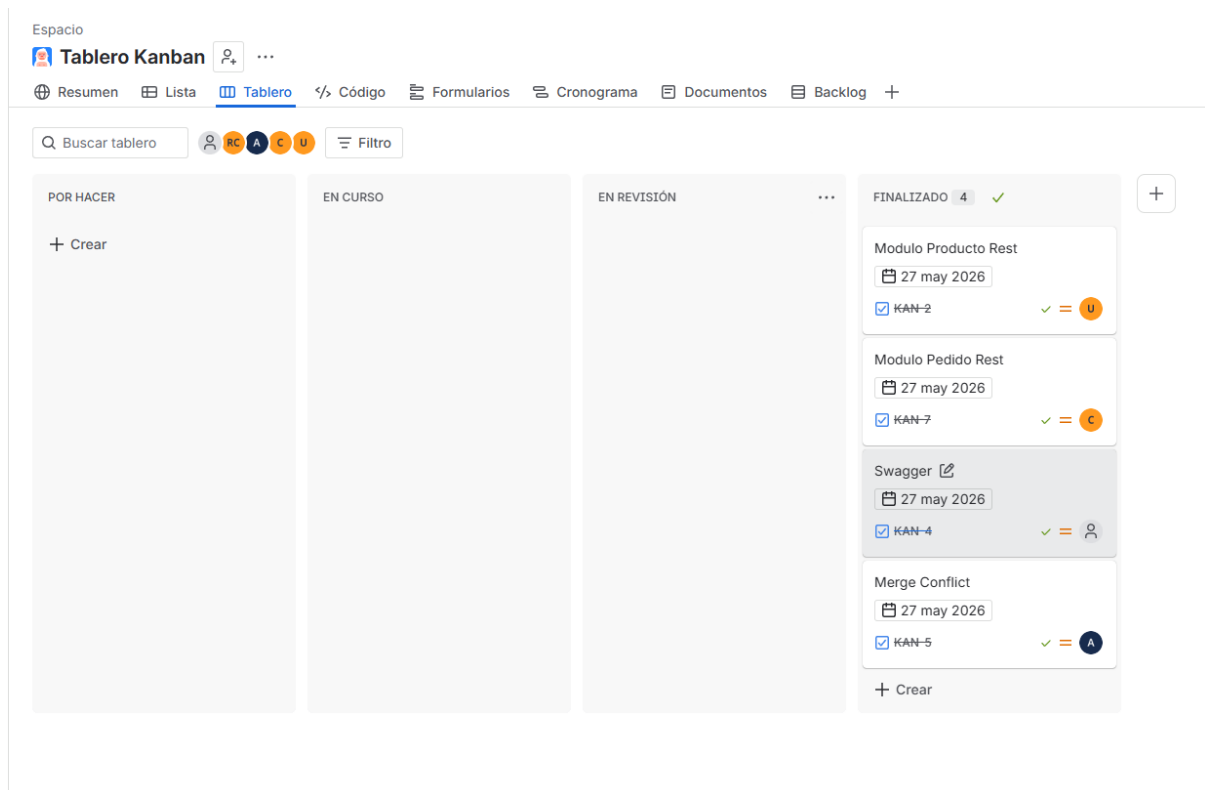
3325686 docs: completar informe Merge Conflict con evidencias
* fec1b96 merge: resolver conflicto entre conflicto-A y conflicto-B en feature/conflictos
```

Las dos siguientes imágenes evidencian la evolución y progreso de las tareas dentro del tablero Kanban durante el desarrollo del proyecto.

En la primera imagen, todas las actividades del proyecto se encontraban ubicadas en la columna: Por Hacer. Esto indicó que las tareas ya habían sido creadas, organizadas y asignadas a los integrantes correspondientes, pero aún no se iniciaba el desarrollo de cada módulo. Entre las actividades registradas se encontraban el módulo Producto REST, el módulo Pedido REST, la integración de Swagger y la resolución de Merge Conflicts.



Esta etapa representó la fase inicial de planificación y organización del proyecto. Posteriormente, en la segunda imagen, se evidenció un avance significativo dentro del flujo de trabajo del tablero Kanban. Todas las tareas fueron trasladadas a la columna “Finalizado”, lo que demostró que cada actividad fue desarrollada, revisada y completada correctamente. Además, la columna “Por Hacer” quedó vacía, demostrando que no quedaron actividades pendientes dentro del proyecto.



El cambio entre ambas imágenes representó el progreso del proyecto desde la etapa de planificación hasta la finalización de las actividades. Gracias al uso del tablero Kanban en Jira, fue posible visualizar el estado de cada tarea, organizar el trabajo colaborativo y realizar un seguimiento adecuado del avance del sistema. Asimismo, esta metodología permitió mejorar la coordinación entre los integrantes y mantener un control visual del desarrollo del proyecto en cada una de sus etapas.